

Задание на учебную практику для студентов направления 09.03.04

Программная инженерия

Разработка интеллектуальной поисковой системы по внутренней базе знаний университета

Срок проведения: «23» июня 2026 г. — «06» июля 2026 г. (14 календарных дней)

Форма отчетности Презентация проекта + Исходный код в репозитории +
Техническая документация + Дневник по практике

1. ОБЩИЕ ПОЛОЖЕНИЯ

Цель практики

Закрепление и углубление теоретических знаний, полученных при изучении дисциплин через разработку полнофункционального программного продукта с использованием современных технологий.

Задачи практики

1. Разработать архитектуру приложения на основе микросервисного подхода.
2. Реализовать систему полнотекстового поиска с использованием Elasticsearch.
3. Создать пользовательский интерфейс для загрузки документов и поиска по ним.
4. Настроить среду исполнения с использованием контейнеризации.
5. Автоматизировать процесс тестирования и развертывания.
6. Обеспечить документирование разработанного программного обеспечения.

Состав команды и распределение ролей

Каждая команда состоит из 4 студентов. Распределение ролей осуществляется на первом организационном собрании.

№	Роль	Код роли	Функциональные обязанности
1.	Бэкенд-инженер	BE	Разработка API, интеграция с Elasticsearch, бизнес-логика, работа с базами данных
2.	Фронтенд-инженер	FE	Разработка пользовательского интерфейса, верстка, клиент-серверное взаимодействие
3.	DevOps-инженер	DO	Контейнеризация, CI/CD настройка, мониторинг, управление окружением
4.	QA-инженер	QA	Автоматизация тестирования, подготовка тестовых данных, контроль качества, документация

2. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ К СИСТЕМЕ

Общие требования

Система должна представлять собой веб-приложение, обеспечивающее:

- Авторизацию пользователей по упрощенной схеме (логин/пароль, без сложных ролей, либо вовсе без авторизации для прототипа).

- Загрузку документов в форматах PDF и DOCX.
- Полнотекстовый поиск по загруженным документам с ранжированием результатов.
- Отображение найденных фрагментов с подсветкой совпадений.
- Сохранение истории поисковых запросов.

Требования к бэкенд-части

Модуль загрузки и обработки документов

ID требования	Описание	Приоритет
BE-01	Реализовать REST API эндпоинт `POST /api/v1/documents/upload` для загрузки файлов	Высокий
BE-02	Проводить валидацию файлов: формат (PDF, DOCX), размер (не более 20 МБ). При несоответствии возвращать HTTP 400 с описанием ошибки	Высокий
BE-03	Генерировать уникальный идентификатор (UUID) для каждого загруженного документа	Высокий
BE-04	Реализовать извлечение текста из загруженных документов с использованием библиотек: pdfplumber (для PDF), python-docx (для DOCX)	Высокий
BE-05	Выполнять разбиение извлеченного текста на чанки (сегменты) по 1000 символов с перекрытием 100 символов между соседними чанками	Средний

Модуль индексации и поиска

ID требования	Описание	Приоритет
BE-06	Настроить Elasticsearch с русскоязычным анализатором (analysis-ru). Создать индекс documents с маппингом для полнотекстового поиска	Высокий
BE-07	Реализовать функцию индексации: каждый чанк документа должен быть сохранен в Elasticsearch с метаданными (file_name, page number, chunk id, text)	Высокий
BE-08	Реализовать эндпоинт GET /api/v1/search?q={query}. Запрос должен выполняться к Elasticsearch с использованием multi match по полю text	Высокий
BE-09	Результаты поиска должны возвращаться в формате JSON с полями: chunk_id, file_name, page, text, score (релевантность)	Высокий

BE-10	Реализовать кэширование частых запросов через Redis (TTL = 5 минут). Если один и тот же запрос поступает повторно, ответ должен браться из кеша	Низкий
-------	---	--------

Требования к API

- Все эндпоинты должны быть задокументированы через OpenAPI 3.0 (Swagger UI доступен по /docs).
- Ответы должны содержать корректные HTTP-статусы:
 - 200 OK — успешное выполнение.
 - 400 Bad Request — ошибка валидации.
 - 404 Not Found — ресурс не найден.
 - 500 Internal Server Error — внутренняя ошибка сервера.

Требования к фронтенд-части

Интерфейс загрузки документов

ID требования	Описание	Приоритет
FE-01	Главная страница должна содержать область для перетаскивания файлов (Drag-and-Drop) с поддержкой множественной загрузки	Высокий
FE-02	Отображать прогресс загрузки и индексации каждого файла в виде прогресс-бара. Состояния: Загрузка..., Индексация..., Готово, Ошибка	Высокий
FE-03	Отображать список уже загруженных документов с их названиями, датой загрузки и статусом	Средний

Интерфейс поиска

ID требования	Описание	Приоритет
FE-04	Реализовать поле ввода поискового запроса с кнопкой «Найти». Поиск должен запускаться как по кнопке, так и по нажатию клавиши Enter	Высокий
FE-05	Результаты поиска выводить в виде карточек. В каждой карточке должны быть: название файла, номер страницы, найденный фрагмент текста, оценка релевантности	Высокий
FE-06	Реализовать подсветку совпадений: все слова, совпавшие с поисковым запросом, должны быть выделены желтым фоном	Высокий

FE-07	Реализовать пагинацию (по 10 результатов на страницу) либо бесконечный скролл (загрузка новых результатов при скролле вниз)	Средний
FE-08	При отсутствии результатов выводить сообщение: «По вашему запросу ничего не найдено. Попробуйте изменить формулировку»	Низкий
FE-09	Верстка должна быть адаптивной (корректно отображаться на экранах от 320px до 1920px)	Средний

Требования к DevOps-части

ID требования	Описание	Приоритет
DO-01	Разработать Dockerfile для сборки образа бэкенд-приложения (Python + FastAPI)	Высокий
DO-02	Разработать Dockerfile для сборки образа фронтенд-приложения (Node.js + React/Vue с последующей отдачей статики через Nginx)	Высокий
DO-03	Написать docker-compose.yml для запуска следующих сервисов: бэкенд (app), фронтенд (front), PostgreSQL (postgres), Elasticsearch (elasticsearch), Redis (redis).	Высокий
DO-04	Все конфиденциальные данные (пароли к БД, ключи) должны передаваться через переменные окружения (файл .env). Файл .env.example должен быть в репозитории	Высокий
DO-05	Настроить GitHub Actions CI/CD: при пуше в ветку main автоматически запускать линтеры и базовые тесты. В случае успеха — автоматически собирать образы	Средний
DO-06	Настроить базовый мониторинг: подключить Prometheus для сбора метрик (количество запросов к /search, среднее время ответа) и Grafana для отображения дашборда	Низкий
DO-07	Написать скрипт инициализации (init.sh), который скачивает 10 тестовых PDF-лекций из открытого доступа и загружает их в систему через API	Средний

Требования к тестированию (QA)

ID требования	Описание	Приоритет
QA-01	Разработать юнит-тесты для бэкенд-модулей (парсинг документов, валидация) с использованием pytest. Покрытие кода тестами — не менее 50%	Высокий
QA-02	Разработать E2E-тесты с использованием Playwright (или Selenium) для проверки	Высокий

	критических пользовательских сценариев: загрузка документа → индексация → поиск → отображение результатов	
QA-03	Подготовить набор тестовых документов, включающий: корректные PDF/DOCX, пустые файлы, файлы с битым форматированием, файлы с нестандартными шрифтами	Высокий
QA-04	Написать нагрузочные тесты с использованием для имитации 50 одновременных пользователей, выполняющих поисковые запросы. Собрать отчет о времени отклика	Средний
QA-05	Провести оценку качества поиска (метрика Precision@3): для 10 эталонных запросов проверить, что нужный документ входит в топ-3 выдачи. Результаты оформить в виде таблицы	Средний
QA-06	Оформить «Руководство пользователя» в формате PDF (или Markdown) с инструкцией по работе с системой	Низкий

3. ТЕХНИЧЕСКИЕ ОГРАНИЧЕНИЯ

Технологический стек (необязательный к применению, но желательный)

Python, FastAPI, PostgreSQL, Elasticsearch, Redis, TypeScript, React (или Vue 3), Docker / Docker Compose, GitHub Actions, Pytest, OpenAPI (Swagger)

Требования к качеству кода

1. Код должен быть оформлен в соответствии с руководствами по написанию кода (PEP 8 и т.п.)
2. Все публичные функции и классы должны содержать описание, параметры, возвращаемые значения.
3. Названия переменных и функций должны быть понятными и отражать суть выполняемых действий.
4. В репозитории запрещено хранение бинарных файлов (за исключением папки /tests/fixtures для тестовых документов).
5. Каждый коммит должен иметь осмысленное сообщение: feat: add document upload endpoint, fix: correct PDF parsing encoding.

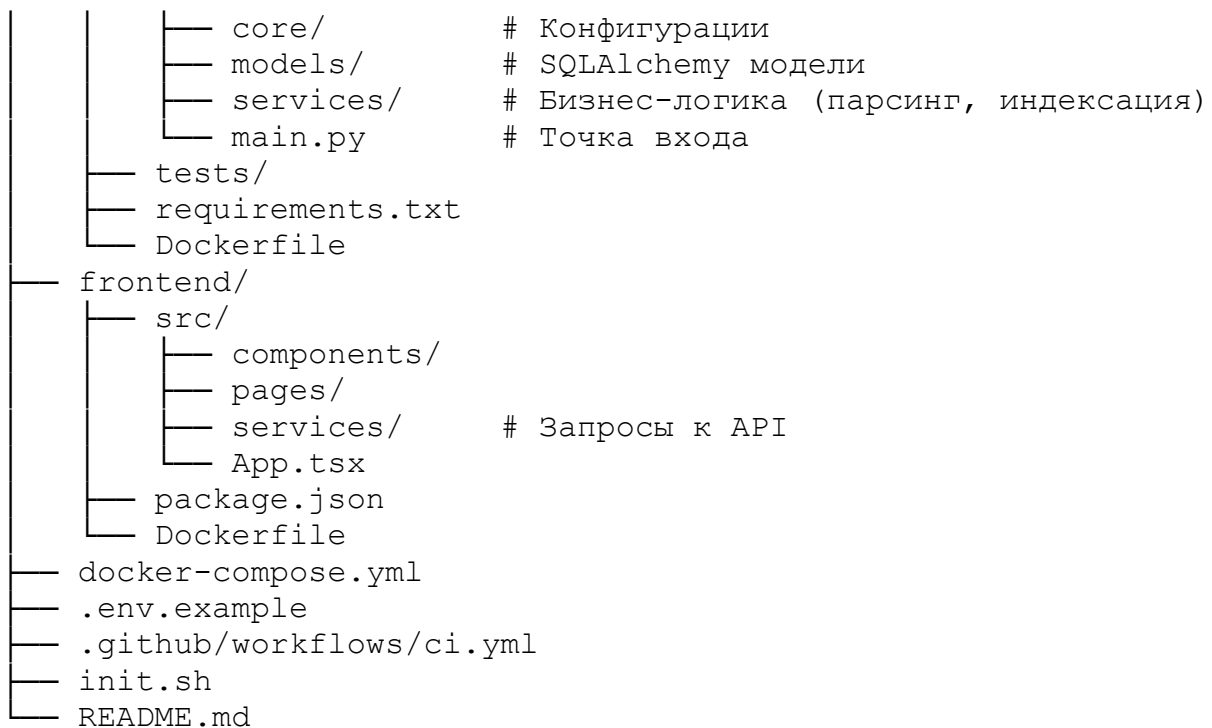
Требования к репозиторию (Git)

Структура репозитория должна соответствовать следующему шаблону:

```

/
├── backend/
│   ├── app/
│   └── api/           # Эндпоинты

```



4. ПОЭТАПНЫЙ ПЛАН РАБОТ (КАЛЕНДАРНЫЙ ГРАФИК)

Неделя 1: Создание базовой архитектуры (Дни 1–6)

День	Задачи для всей команды	Индивидуальные задачи
1	Организационное собрание. Распределение ролей. Создание репозитория. Обсуждение архитектуры	BE: Установка FastAPI, создание main.py. FE: Инициализация React-приложения (Vite). DO: Написание docker-compose.yml для PostgreSQL + Elasticsearch. QA: Изучение документации, подготовка тестовых документов
2	Согласование структуры API (Swagger-спецификация)	BE: Реализация эндпоинта /upload с валидацией. FE: Верстка главной страницы + Drag-and-Drop зона. DO: Написание Dockerfile для бэкенда. QA: Написание тестов для валидации.
3	Синхронизация формата JSON между BE и FE	BE: Интеграция с Elasticsearch (создание индекса). FE: Реализация списка загруженных документов. DO: Написание Dockerfile для фронтенда. QA: Написание E2E-теста загрузки файла.
4	Проверка работы парсера на тестовых файлах	BE: Реализация парсинга PDF и DOCX. FE: Стилизация компонентов (CSS/SCSS). DO: Настройка GitHub Actions (линтеры). QA: Подготовка бага-репортов по парсингу.
5	День интеграции №1. Стыковка фронта и бэка через локальный Docker Compose	Все: Фикс ошибок CORS, несоответствий полей. Запуск полного стека
6	Заполнение отчетных документов	Все: заполнение дневника и подготовка отчета.

Неделя 2: Функциональное наполнение и сдача (Дни 6–10)

День	Задачи для всей команды	Индивидуальные задачи
7	Согласование логики поиска и подсветки	BE: Реализация эндпоинта /search с Elasticsearch. FE: Верстка карточек результатов поиска. DO: Настройка Redis для кеширования. QA: Написание тестов для поиска.
8	Проверка производительности поиска	BE: Оптимизация запросов к ES (фильтры) FE: Реализация подсветки совпадений DO: Интеграция Prometheus + Grafana QA: Запуск нагрузочных тестов
9	День интеграции №2. Полное тестирование сквозных сценариев	BE: Добавление пагинации в API. FE: Реализация бесконечного скролла/пагинации. DO: Доработка CI/CD (автодеплой на сервер). QA: Оценка Precision@3, написание отчета
10	Полировка UI и исправление критических багов	BE: Документирование кода. FE: Адаптивная верстка под мобильные устройства. DO: Написание README.md и init.sh. QA: Оформление User Guide
11	Заполнение отчетных документов	Все: заполнение дневника и подготовка отчета.
12	Предзащита. Подготовка презентации (5–7 слайдов). Демонстрация работы системы. Сдача отчета.	Все: Проверка выполнения всех критериев. Фиксация итоговых метрик

5. КРИТЕРИИ ОЦЕНИВАНИЯ

Общая оценка

Функциональность - Выполнение всех требований из Раздела 2. Корректная работа загрузки, поиска, подсветки.

Качество кода - Соответствие стандартам, наличие документации, чистота архитектуры.

DevOps - Работа Docker Compose, успешный запуск одной командой, наличие CI/CD.

Тестирование - Наличие юнит-тестов, E2E-тестов, нагрузочных тестов, покрытие > 50%.

Документация - Наличие README, User Guide, Swagger-документации.

Презентация - Качество защиты проекта, ответы на вопросы.